

Universidad de Costa Rica

Laboratorio de Microcontroladores

Laboratorio #1:
Introducción a microcontroladores y manejo de GPIOs

Prof. MSc. Marco Villalta Fallas

Marco Vásquez Ovarés

B17032

Grupo: 01

I Ciclo 2025

Índice

1. Introducción	5
2. Nota teórica	6
2.1. Características generales	7
2.2. Diagrama de pines	8
2.3. Diagrama de bloques	10
2.4. Mapa de memoria de datos	11
2.5. Características eléctricas	11
2.6. Periféricos	12
2.6.1. Registros	12
2.6.2. Registro GPIO	12
2.6.3. Registro TRISIO	12
2.6.4. Registro ANSEL	13
2.6.5. Registro CMCON0	13
2.6.6. Configuración CONFIG	13
2.7. Componentes electrónicos complementarios	14
2.7.1. Leds	14
2.7.2. Switch	15
2.7.3. Botón	15
2.8. Diseño del circuito	16
2.8.1. Lista de componentes	16
2.8.2. Información adicional	16
3. Desarrollo	20
3.1. Análisis del firmware	20
3.1.1. Configuración del microcontrolador	20
3.1.2. Definición del botón y funciones	20
3.1.3. Función main()	20
3.1.4. Función dice()	20
3.1.5. Función de retardo (delay())	22
3.2. Análisis electrónico	22
3.2.1. Implementación del boton	22
3.2.2. Iluminación de los leds	24
3.3. Análisis de resultados	24
4. Conclusiones y Recomendaciones	26
4.1. Conclusiones	26
4.2. Recomendaciones	26
Bibliografía	27
5. Apéndices	27
5.1. Repositorio Git	27
5.2. Outputs	27
5.2.1. Cara 1 del Dado	28

5.2.2.	Cara 2 del Dado		29
5.2.3.	Cara 3 del Dado		30
5.2.4.	Cara 4 del Dado		31
5.2.5.	Cara 5 del Dado		32
5.2.6.	Cara 6 del Dado		33

Índice de figuras

1.	Microcontrolador PIC12F683	6
2.	Diagrama de Pines, Microcontrolador PIC12F683	8
3.	Resumen Diagrama de Pines, Microcontrolador PIC12F683	8
4.	Descripción de Pines, Microcontrolador PIC12F683	9
5.	Diagrama de bloques, Microcontrolador PIC12F683	10
6.	Mapa de memoria de datos, Microcontrolador PIC12F683	11
7.	Características eléctricas, Microcontrolador PIC12F683	12
8.	GPIO: GENERAL PURPOSE I/O REGISTER	12
9.	TRISIO GPIO TRI-STATE REGISTER	13
10.	ANSEL: ANALOG SELECT REGISTER	13
11.	CMCON0: COMPARATOR CONFIGURATION REGISTER	13
12.	CONFIG	14
13.	Leds	14
14.	Led amarillo	15
15.	Boton	16
16.	Diseño final del circuito	16
17.	Lectura digital - Pull-up	17
18.	Lectura digital - Pull-down	18
19.	Manejo de rebotes	19
20.	Botón Off	23
21.	Botón On	23
22.	Mediciones eléctricas	24
23.	Medición eléctricas, pin0	25
24.	Medición eléctricas, (pin1, pin2, pin4)	25
25.	Caras de un dado	27

Índice de tablas

1.	Lista de componentes	17
----	--------------------------------	----

1. Introducción

Resumen

En este laboratorio se realiza una introducción al uso del microcontrolador PIC12F683, utilizando sus pines como GPIOs (General Purpose Input/Output) para diseñar y programar un simulador de dados con LEDs. El objetivo es lograr que seis LEDs se enciendan de manera pseudoaleatoria entre los valores del 1 al 6 cuando se pulse un botón, manteniendo la iluminación durante un período determinado.

Para desarrollar el simulador, primero se analizó la hoja de datos del microcontrolador y se comprendió el funcionamiento de sus pines. Posteriormente, se escribió el código en lenguaje C para programar el firmware del microcontrolador, considerando aspectos como la configuración de registros y la correcta interacción con los periféricos. Adicionalmente, en conjunto, se simula el circuito con los componentes necesarios, asegurando la protección tanto del microcontrolador como de los LEDs.

Durante la práctica, se estudiaron diversos conceptos clave, incluyendo el efecto rebote o "bouncing", un fenómeno que genera múltiples transiciones no deseadas al presionar el botón. Este problema se mitigó utilizando un resistor y un capacitor para estabilizar la señal de entrada.

El desarrollo del laboratorio se llevó a cabo en el simulador SimulIDE, permitiendo probar y validar el funcionamiento del circuito antes de una implementación física. Como resultado, se logró un dado electrónico funcional que demuestra la capacidad de los microcontroladores para gestionar entradas y salidas digitales de manera eficiente, proporcionando una base sólida para futuros proyectos con PICs.

2. Nota teórica

A continuación se brinda información relevante acerca del microcontrolador PIC, en concreto sobre el PIC12F683, así como las características, periféricos, entre otros.

El PIC12F683 es un microcontrolador de 8 bits de la familia PIC de Microchip Technology, diseñado para aplicaciones de bajo consumo y tamaño reducido. Cuenta con un núcleo RISC optimizado que permite una ejecución eficiente de instrucciones.

Características principales:

- Arquitectura: Harvard de 8 bits
- Memoria Flash: 2 KB
- RAM: 128 bytes
- EEPROM: 256 bytes (para almacenamiento no volátil)
- Velocidad: Hasta 20 MHz con oscilador externo
- Puertos de I/O: 6 pines de propósito general
- Módulos integrados:
 - Convertidor ADC de 10 bits
 - Comparador analógico
 - Módulo PWM
 - Temporizadores (Timer0, Timer1 y Timer2)
 - Comunicación USART/Serial mediante bit-banging



Figura 1: Microcontrolador PIC12F683

2.1. Características generales

El PIC12F683 es un microcontrolador de 8 bits de bajo consumo con una arquitectura avanzada y módulos integrados que lo hacen versátil para diversas aplicaciones:

Arquitectura y memoria

- Arquitectura: Harvard de 8 bits con conjunto de instrucciones RISC
- Memoria Flash: 2 KB (para almacenamiento de programa)
- Memoria RAM: 128 bytes (para almacenamiento temporal)
- Memoria EEPROM: 256 bytes (para almacenamiento no volátil)
- Ciclo de instrucción: 4 ciclos de reloj por instrucción

Entradas y salidas (I/O)

- Número de pines: 8 pines en total
- Pines de I/O: 6 pines configurables como entrada o salida digital
- Corriente máxima por pin: 25 mA

Periféricos integrados

- ADC (Convertidor Analógico-Digital) de 10 bits con 4 canales
- PWM (Modulación por Ancho de Pulso) con 1 canal
- Comparador analógico con referencia interna o externa
- Timer0, Timer1 y Timer2 para control de tiempo y generación de eventos
- Módulo CCP (Capture/Compare/PWM)

Reloj y velocidad de operación

- Frecuencia máxima: 20 MHz con oscilador externo
- Oscilador interno: 8 MHz ajustable
- Modos de bajo consumo: Sleep y Watchdog Timer

Comunicación y funciones especiales

- Comunicación serie: Implementable por software (bit-banging)
- Interrupciones: Soporte para interrupciones internas y externas
- Protección contra fallas: Watchdog Timer y Brown-out Reset (BOR)

Consumo de energía

- Modos de bajo consumo: Sleep mode y Wake-up por interrupción
- Voltaje de operación: 2.0V - 5.5V
- Consumo en operación: Bajo consumo en comparación con otros microcontroladores de 8 bits

Empaquetado disponible

- DIP de 8 pines
- SOIC y DFN para aplicaciones en espacios reducidos

2.2. Diagrama de pines

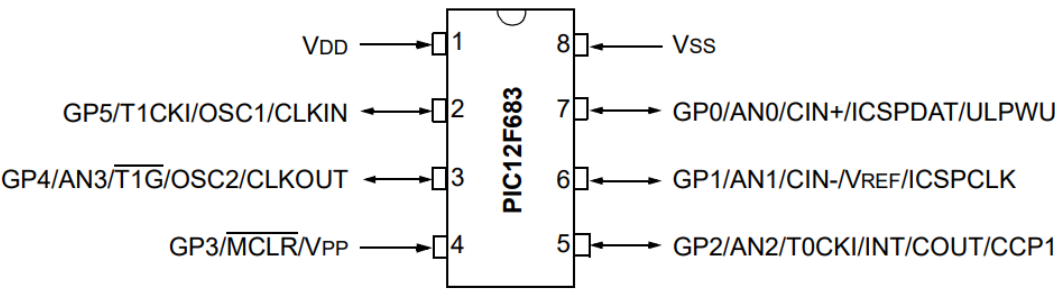


Figura 2: Diagrama de Pines, Microcontrolador PIC12F683

I/O	Pin	Analog	Comparators	Timer	CCP	Interrupts	Pull-ups	Basic
GP0	7	AN0	CIN+	—	—	IOC	Y	ICSPDAT/ULPWU
GP1	6	AN1/VREF	CIN-	—	—	IOC	Y	ICSPCLK
GP2	5	AN2	COUT	T0CKI	CCP1	INT/IOC	Y	—
GP3 ⁽¹⁾	4	—	—	—	—	IOC	Y ⁽²⁾	MCLR/VPP
GP4	3	AN3	—	T1G	—	IOC	Y	OSC2/CLKOUT
GP5	2	—	—	T1CKI	—	IOC	Y	OSC1/CLKIN
—	1	—	—	—	—	—	—	VDD
—	8	—	—	—	—	—	—	Vss

Note 1: Input only.
2: Only when pin is configured for external MCLR.

Figura 3: Resumen Diagrama de Pines, Microcontrolador PIC12F683

Name	Function	Input Type	Output Type	Description
VDD	VDD	Power	—	Positive supply
GP5/T1CKI/OSC1/CLKIN	GP5	TTL	CMOS	GPIO I/O with prog. pull-up and interrupt-on-change
	T1CKI	ST	—	Timer1 clock
	OSC1	XTAL	—	Crystal/Resonator
	CLKIN	ST	—	External clock input/RC oscillator connection
GP4/AN3/T1G/OSC2/CLKOUT	GP4	TTL	CMOS	GPIO I/O with prog. pull-up and interrupt-on-change
	AN3	AN	—	A/D Channel 3 input
	T1G	ST	—	Timer1 gate
	OSC2	—	XTAL	Crystal/Resonator
	CLKOUT	—	CMOS	Fosc/4 output
GP3/MCLR/VPP	GP3	TTL	—	GPIO input with interrupt-on-change
	MCLR	ST	—	Master Clear with internal pull-up
	VPP	HV	—	Programming voltage
GP2/AN2/T0CKI/INT/COUT/CCP1	GP2	ST	CMOS	GPIO I/O with prog. pull-up and interrupt-on-change
	AN2	AN	—	A/D Channel 2 input
	T0CKI	ST	—	Timer0 clock input
	INT	ST	—	External Interrupt
	COUT	—	CMOS	Comparator 1 output
	CCP1	ST	CMOS	Capture input/Compare output/PWM output
GP1/AN1/CIN-/VREF/ICSPCLK	GP1	TTL	CMOS	GPIO I/O with prog. pull-up and interrupt-on-change
	AN1	AN	—	A/D Channel 1 input
	CIN-	AN	—	Comparator 1 input
	VREF	AN	—	External Voltage Reference for A/D
	ICSPCLK	ST	—	Serial Programming Clock
GP0/AN0/CIN+/ICSPDAT/ULPWU	GP0	TTL	CMOS	GPIO I/O with prog. pull-up and interrupt-on-change
	AN0	AN	—	A/D Channel 0 input
	CIN+	AN	—	Comparator 1 input
	ICSPDAT	ST	CMOS	Serial Programming Data I/O
	ULPWU	AN	—	Ultra Low-Power Wake-up input
VSS	VSS	Power	—	Ground reference

Legend:

AN = Analog input or output	CMOS = CMOS compatible input or output
TTL = TTL compatible input	ST = Schmitt Trigger input with CMOS levels
HV = High Voltage	XTAL = Crystal

Figura 4: Descripción de Pines, Microcontrolador PIC12F683

2.3. Diagrama de bloques

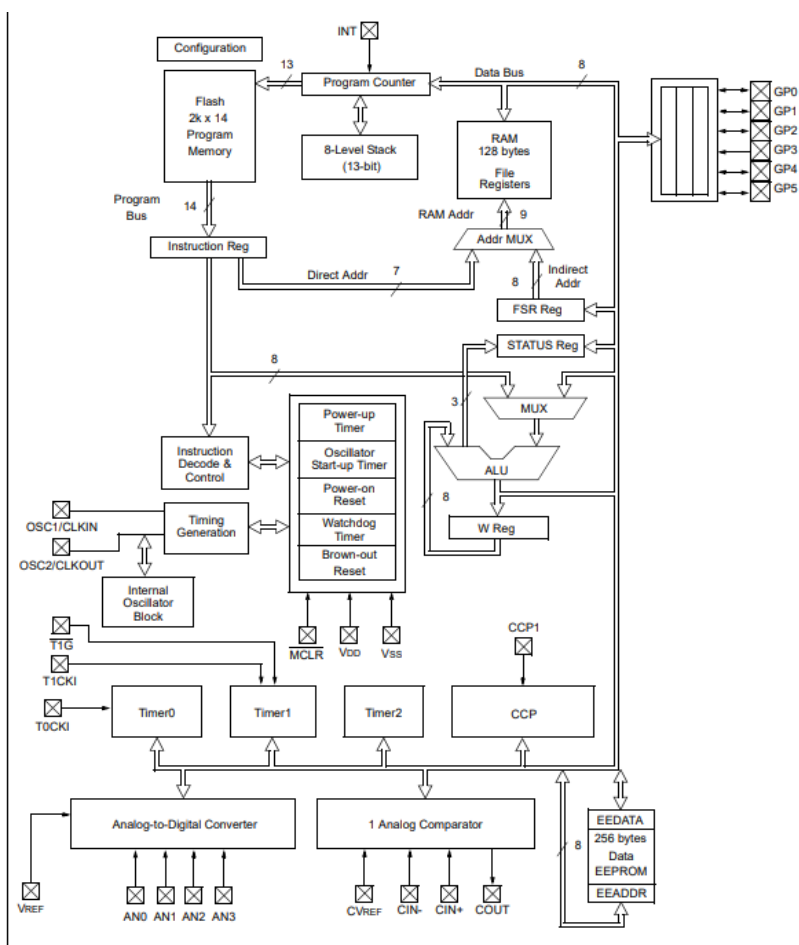


Figura 5: Diagrama de bloques, Microcontrolador PIC12F683

2.4. Mapa de memoeria de datos



Figura 6: Mapa de memoria de datos, Microcontrolador PIC12F683

2.5. Características eléctricas

Esta sección es de especial interés para trabajar con el microcontrolador, es importante tener en consideración sus características eléctricas para llevar a cabo las prácticas de laboratorio y evitar

fallos; independientemente de si se trabaja con una simulación o en físico es bueno respetar los rangos de operación del microcontrolador.

Absolute Maximum Ratings^(†)

Ambient temperature under bias.....	-40° to +125°C
Storage temperature	-65°C to +150°C
Voltage on VDD with respect to VSS	-0.3V to +6.5V
Voltage on MCLR with respect to VSS	-0.3V to +13.5V
Voltage on all other pins with respect to VSS	-0.3V to (VDD + 0.3V)
Total power dissipation ^(†)	800 mW
Maximum current out of VSS pin	95 mA
Maximum current into VDD pin	95 mA
Input clamp current, I _{IK} (V _I < 0 or V _I > VDD).....	± 20 mA
Output clamp current, I _{OK} (V _O < 0 or V _O > VDD).....	± 20 mA
Maximum output current sunk by any I/O pin.....	25 mA
Maximum output current sourced by any I/O pin	25 mA
Maximum current sunk by GPIO	90 mA
Maximum current sourced GPIO.....	90 mA

Figura 7: Características eléctricas, Microcontrolador PIC12F683

2.6. Periféricos

2.6.1. Registros

En el PIC12F683, la configuración y control de los pines de entrada/salida (I/O) se realiza a través de registros específicos. A continuación, se describen los registros más importantes para este propósito:

2.6.2. Registro GPIO

- Controla el estado lógico de los pines de entrada/salida.
- Es un registro de 6 bits (GPIO0 a GPIO5), ya que el PIC12F683 tiene 6 pines de I/O utilizables.
- Ejemplo: GPIO = 0b000001 enciende el pin GPIO0 en alto (1), mientras que los demás quedan en bajo (0).

U-0	U-0	R/W-x	R/W-0	R-x	R/W-0	R/W-0	R/W-0
—	—	GP5	GP4	GP3	GP2	GP1	GP0
bit 7		bit 0					

Figura 8: GPIO: GENERAL PURPOSE I/O REGISTER

2.6.3. Registro TRISIO

- Configura cada pin como entrada (1) o salida (0).
- Un bit en 1 convierte el pin en entrada, y un bit en 0 lo configura como salida.

- Ejemplo: TRISIO = 0b000011 configura GPIO0 y GPIO1 como entradas y los demás como salidas.

U-0	U-0	R/W-1	R/W-1	R-1	R/W-1	R/W-1	R/W-1
—	—	TRISIO5 ^(2,3)	TRISIO4 ⁽²⁾	TRISIO3 ⁽¹⁾	TRISIO2	TRISIO1	TRISIO0
bit 7							bit 0

Figura 9: TRISIO GPIO TRI-STATE REGISTER

2.6.4. Registro ANSEL

- Determina si los pines funcionan en modo analógico (ADC) o digital.
- Un bit en 1 habilita la función analógica, mientras que 0 lo deja como digital.
- Ejemplo: ANSEL = 0b00000001 habilita el modo analógico en GPIO0 y deja los demás en digital.

U-0	R/W-0	R/W-0	R/W-0	R/W-1	R/W-1	R/W-1	R/W-1
—	ADCS2	ADCS1	ADCS0	ANS3	ANS2	ANS1	ANS0
bit 7							bit 0

Figura 10: ANSEL: ANALOG SELECT REGISTER

2.6.5. Registro CMCON0

El registro CMCON0 controla el comparador analógico integrado en el PIC12F683. Los comparadores permiten comparar señales analógicas y generar una salida digital basada en la comparación.

U-0	R-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
—	COUT	—	CINV	CIS	CM2	CM1	CM0
bit 7							bit 0

Figura 11: CMCON0: COMPARATOR CONFIGURATION REGISTER

2.6.6. Configuración CONFIG

Contiene configuraciones globales del microcontrolador

```

#define _CONFIG      0x2007

//----- CONFIG Options -----

#define _FOSC_LP      0x3FF8 // LP oscillator: Low-power crystal on RA4/OSC2/CLKOUT and RA5/OSC1/CLKIN.
#define _LP_OSC      0x3FF8 // LP oscillator: Low-power crystal on RA4/OSC2/CLKOUT and RA5/OSC1/CLKIN.
#define _FOSC_XT      0x3FF9 // XT oscillator: Crystal/resonator on RA4/OSC2/CLKOUT and RA5/OSC1/CLKINT.
#define _XT_OSC      0x3FF9 // XT oscillator: Crystal/resonator on RA4/OSC2/CLKOUT and RA5/OSC1/CLKINT.
#define _FOSC_HS      0x3FFA // HS oscillator: High-speed crystal/resonator on RA4/OSC2/CLKOUT and RA5/OSC1/CLKIN.
#define _HS_OSC      0x3FFA // HS oscillator: High-speed crystal/resonator on RA4/OSC2/CLKOUT and RA5/OSC1/CLKIN.
#define _FOSC_EC      0x3FFB // EC: I/O function on RA4/OSC2/CLKOUT pin, CLKIN on RA5/OSC1/CLKIN.
#define _EC_OSC      0x3FFB // EC: I/O function on RA4/OSC2/CLKOUT pin, CLKIN on RA5/OSC1/CLKIN.
#define _FOSC_INTOSCIO 0x3FFC // INTOSCIO oscillator: I/O function on RA4/OSC2/CLKOUT pin, I/O function on RA5/OSC1/CLKIN.
#define _INTOSCIO     0x3FFC // INTOSCIO oscillator: I/O function on RA4/OSC2/CLKOUT pin, I/O function on RA5/OSC1/CLKIN.
#define _INTRC_OSC_NOCLKOUT 0x3FFC // INTOSCIO oscillator: I/O function on RA4/OSC2/CLKOUT pin, I/O function on RA5/OSC1/CLKIN.
#define _FOSC_INTOSCCLK 0x3FFD // INTOSC oscillator: CLKOUT function on RA4/OSC2/CLKOUT pin, I/O function on RA5/OSC1/CLKIN.
#define _INTOSC       0x3FFD // INTOSC oscillator: CLKOUT function on RA4/OSC2/CLKOUT pin, I/O function on RA5/OSC1/CLKIN.
#define _INTRC_OSC_CLKOUT 0x3FFD // INTOSC oscillator: CLKOUT function on RA4/OSC2/CLKOUT pin, I/O function on RA5/OSC1/CLKIN.
#define _FOSC_EXTRCIO 0x3FFE // EXTRCIO oscillator: External RC on RA5/OSC1/CLKIN, I/O function on RA4/OSC2/CLKOUT pin.
#define _EXTRCIO      0x3FFE // EXTRCIO oscillator: External RC on RA5/OSC1/CLKIN, I/O function on RA4/OSC2/CLKOUT pin.
#define _EXTRC_OSC_NOCLKOUT 0x3FFE // EXTRCIO oscillator: External RC on RA5/OSC1/CLKIN, I/O function on RA4/OSC2/CLKOUT pin.
#define _EXTRC        0x3FFE // EXTRC oscillator: External RC on RA5/OSC1/CLKIN, CLKOUT function on RA4/OSC2/CLKOUT pin.
#define _FOSC_EXTRCCLK 0x3FFF // EXTRC oscillator: External RC on RA5/OSC1/CLKIN, CLKOUT function on RA4/OSC2/CLKOUT pin.
#define _EXTRC        0x3FFF // EXTRC oscillator: External RC on RA5/OSC1/CLKIN, CLKOUT function on RA4/OSC2/CLKOUT pin.
#define _EXTRC_OSC_CLKOUT 0x3FFF // EXTRC oscillator: External RC on RA5/OSC1/CLKIN, CLKOUT function on RA4/OSC2/CLKOUT pin.
#define _WDT_OFF      0x3FF7 // WDT disabled.
#define _WDT_OFF      0x3FF7 // WDT disabled.
#define _WDT_ON       0x3FFF // WDT enabled.
#define _WDT_ON       0x3FFF // WDT enabled.
#define _PWRT_ON      0x3FEF // PWRT enabled.
#define _PWRT_ON      0x3FEF // PWRT enabled.
#define _PWRT_OFF     0x3FFF // PWRT disabled.
#define _MCLR_OFF     0x3FDF // MCLR pin function is digital input, MCLR internally tied to VDD.
#define _MCLR_ON      0x3FFF // MCLR pin function is MCLR.
#define _CP_ON        0x3FBF // Program memory code protection is enabled.
#define _CP_OFF       0x3FFF // Program memory code protection is disabled.
#define _CPD_ON       0x3F7F // Data memory code protection is enabled.
#define _CPD_OFF      0x3FFF // Data memory code protection is disabled.
#define _BOREN_OFF    0x3CFF // BOR disabled.
#define _BOD_OFF      0x3CFF // BOR disabled.
#define _BODEN        0x3DFF // BOR controlled by SBODEN bit of the PCON register.
#define _BODEN        0x3DFF // BOR controlled by SBODEN bit of the PCON register.
#define _BODEN_NSLEEP 0x3EFF // BOR enabled during operation and disabled in Sleep.
#define _BODEN_NSLEEP 0x3EFF // BOR enabled during operation and disabled in Sleep.
#define _BODEN_ON     0x3FFF // BOR enabled.
#define _BOD_ON       0x3FFF // BOR enabled.
#define _IESO_OFF     0x3BFF // Internal External Switchover mode is disabled.
#define _IESO_ON      0x3FFF // Internal External Switchover mode is enabled.
#define _FCMEN_OFF    0x37FF // Fail-Safe Clock Monitor is disabled.
#define _FCMEN_ON     0x3FFF // Fail-Safe Clock Monitor is enabled.

//-----

```

Figura 12: CONFIG

2.7. Componentes electrónicos complementarios

2.7.1. Leds

Los LEDs (diodos emisores de luz) son componentes electrónicos que emiten luz cuando la corriente eléctrica pasa a través de ellos. Estos dispositivos son altamente eficientes en términos de energía, tienen una vida útil más prolongada y generan menos calor que las bombillas tradicionales. Los LEDs funcionan mediante el principio de electroluminiscencia, donde un material semiconductor emite fotones cuando es excitado por la corriente eléctrica. Se utilizan en diversas aplicaciones, como indicadores luminosos, pantallas, señalización y sistemas de iluminación.



Figura 13: Leds

El LED amarillo es un tipo de LED que emite luz en el espectro de color amarillo, comúnmente utilizado en aplicaciones que requieren señalización o luz distintiva. Su funcionamiento es similar al de otros LEDs, pero utiliza materiales semiconductores que producen una longitud de onda correspondiente al color amarillo. Los rangos de operación típicos para un LED amarillo son:

- Tensión de operación (Forward Voltage): Aproximadamente entre 2.0V y 2.2V.
- Corriente de operación (Forward Current): Normalmente alrededor de 20 mA.
- Potencia de salida: Dependiendo de la corriente y el tipo de LED, puede variar entre 1.5 mW y 3 mW.
- Rango espectral: La longitud de onda del color amarillo se encuentra cerca de 590 nm.



Figura 14: Led amarillo

2.7.2. Switch

Los switches son interruptores que operan de manera similar a los botones, pero generalmente están diseñados para mantenerse en una posición fija después de ser activados. Un switch puede ser de tipo deslizante, basculante o de palanca, y se utiliza para cambiar el estado de un circuito, encender o apagar dispositivos, o seleccionar entre diferentes opciones. A diferencia de los botones, que vuelven a su estado original cuando se dejan de presionar, los switches mantienen la posición hasta que se cambian manualmente. Al igual que los botones, los switches también pueden experimentar rebote en sus contactos, lo que puede solucionarse mediante técnicas de debouncing.

2.7.3. Botón

Los botones son componentes electrónicos utilizados como interruptores para activar o desactivar un circuito al ser presionados. Se emplean en dispositivos de entrada para cambiar el estado de un sistema, como iniciar un proceso, encender una luz o alternar entre diferentes modos. Sin embargo, debido al efecto rebote (switch bounce), los botones pueden generar múltiples transiciones rápidas entre alto y bajo al presionarse, lo que podría resultar en lecturas erróneas. Para contrarrestar este fenómeno, se implementan técnicas de debouncing, ya sea en hardware o software, para asegurar que el sistema lea correctamente el estado del botón.



Figura 15: Boton

2.8. Diseño del circuito

A continuación se presenta el diseño final del circuito:

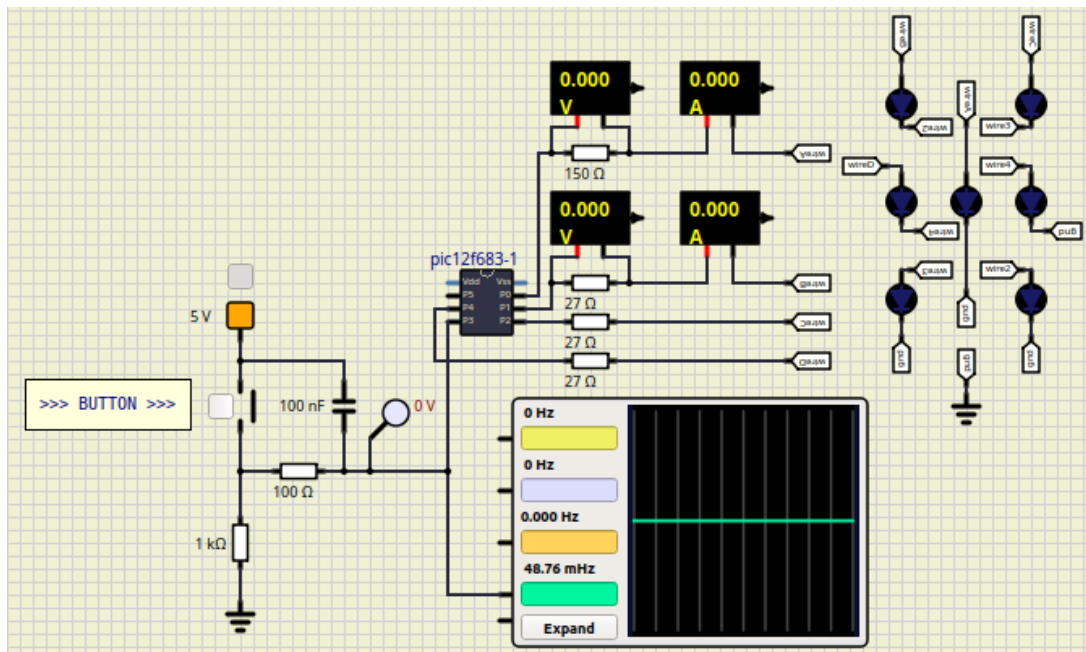


Figura 16: Diseño final del circuito

2.8.1. Lista de componentes

2.8.2. Informacion adicional

GPIO's: Los GPIOs (General Purpose Input/Output) son pines de un microcontrolador configurables como entradas o salidas digitales, permitiendo la interacción con otros dispositivos electrónicos. En modo entrada, los GPIOs leen señales externas, como botones o sensores, mientras que en modo salida envían señales para activar LEDs, motores o relés. Su configuración se realiza mediante registros como TRISx (define si el pin es entrada o salida), PORTx/GPIOn (almacena el estado actual del

Tabla 1: Lista de componentes

Componente	Cantidad	Precio (CRC)
PIC12F683	1	-
Pulsador (Push)	1	-
1k Ω	1	-
100 Ω	1	-
100nF	1	-
150 Ω	1	-
27 Ω	3	-
Leds amarillos	7	-

pin) y en algunos casos LATx (controla la salida de los pines). Los GPIOs son esenciales en sistemas embebidos para el control de dispositivos, lectura de sensores y comunicación con otros periféricos mediante protocolos seriales como I2C, SPI y UART.

Configuración Pull-up: El pull-up es una configuración en la que una resistencia se conecta entre el pin GPIO y el voltaje positivo (Vcc) para asegurar que, cuando no haya una señal activa, el pin se mantenga en un estado lógico alto (1). Se utiliza para evitar estados indefinidos en entradas digitales, especialmente en botones y sensores. Muchos microcontroladores incluyen resistencias pull-up internas que pueden activarse por software, eliminando la necesidad de componentes externos.

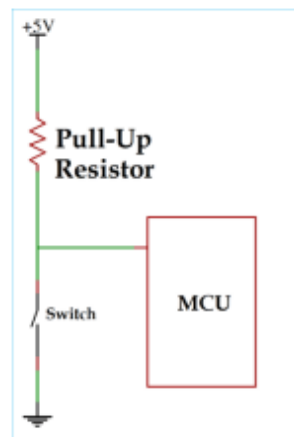


Figura 17: Lectura digital - Pull-up

Configuración Pull-down: El pull-down es una configuración en la que una resistencia se conecta entre el pin GPIO y tierra (GND) para mantener el pin en un estado lógico bajo (0) cuando no haya una señal activa. Se usa en circuitos donde se requiere que el estado por defecto del pin sea bajo, como en entradas de sensores o botones que activan con un nivel alto. A diferencia de los pull-up, no todos los microcontroladores tienen resistencias pull-down internas, por lo que a menudo

se deben colocar externamente.

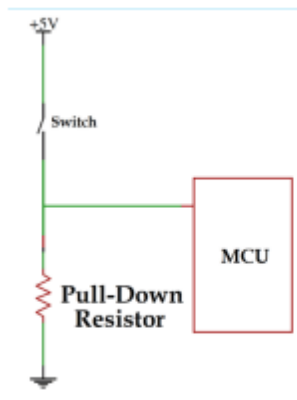


Figura 18: Lectura digital - Pull-down

Efecto rebote (Switch Bounce): El efecto rebote o switch bounce es un fenómeno que ocurre cuando un interruptor o botón mecánico cambia de estado, generando múltiples transiciones rápidas entre alto (1) y bajo (0) antes de estabilizarse. Esto sucede porque los contactos internos del interruptor no cierran o abren instantáneamente, sino que rebotan durante unos milisegundos, lo que puede provocar lecturas erróneas en un microcontrolador.

Para evitar este problema, se aplican técnicas de antirrebote (debouncing), como el uso de retardos por software, filtros electrónicos con capacitores o el empleo de registros de desplazamiento y circuitos lógicos dedicados. También se pueden utilizar interruptores con circuitos internos de eliminación de rebote para aplicaciones críticas.

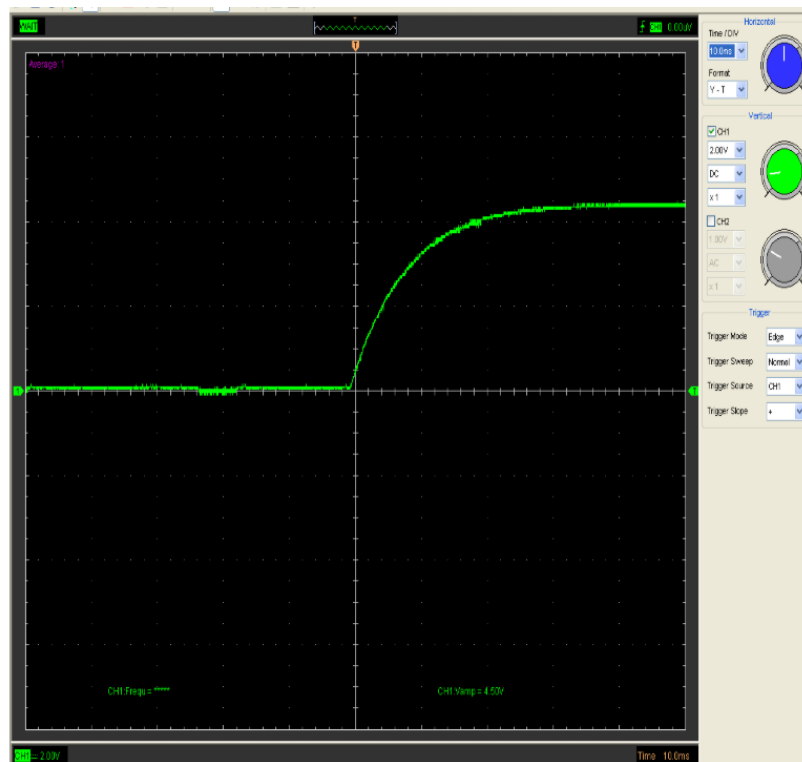


Figura 19: Manejo de rebotes

Generación de números aleatorios: La generación de números aleatorios es un proceso en el que se producen valores impredecibles dentro de un rango determinado. Existen dos tipos principales: los números aleatorios verdaderos (TRNG), obtenidos de fenómenos físicos impredecibles, y los números pseudoaleatorios (PRNG), generados mediante algoritmos matemáticos que imitan la aleatoriedad. En microcontroladores, los PRNG son los más utilizados debido a la falta de fuentes naturales de entropía.

Para generar números aleatorios utilizando un contador interno, se aprovecha la variabilidad del tiempo de ejecución del código o interrupciones asincrónicas. Un método común es leer el valor de un timer (contador interno) en un momento aleatorio, como al presionar un botón o recibir una señal externa, lo que proporciona un número difícil de predecir.

Por otro lado, la generación de números aleatorios por software se basa en algoritmos como el Linear Feedback Shift Register (LFSR) o el congruencial lineal, que producen secuencias de números con apariencia aleatoria a partir de una semilla inicial. Estos métodos son eficientes y pueden implementarse fácilmente en microcontroladores con recursos limitados.

3. Desarrollo

3.1. Análisis del firmware

Este código está escrito para un microcontrolador PIC12F683 y realiza la simulación de un dado utilizando LEDs, donde el valor del dado se genera de manera pseudoaleatoria al presionar un botón conectado al pin GP3. Aquí está el análisis del código:

3.1.1. Configuración del microcontrolador

```
word __at 0x2007 __CONFIG = (_WDT_OFF & _MCLRE_OFF & _BOREN_OFF);
```

Esta línea establece la configuración del microcontrolador:

- `_WDT_OFF`: Apaga el Watchdog Timer (WDT).
- `_MCLRE_OFF`: Deshabilita la función de Master Clear Reset en el pin MCLR, permitiendo que el pin funcione como una entrada digital.
- `_BOREN_OFF`: Desactiva el Brown-out Reset, lo que significa que el microcontrolador no se reiniciará si el voltaje de alimentación cae por debajo de un umbral predefinido.

3.1.2. Definición del botón y funciones

```
#define BTN GP3  
void dice ();  
void delay (unsigned int tiempo);
```

- `BTN` se define como `GP3`, que es el pin de entrada al que está conectado el botón.
- Se declaran las funciones `dice()` (que gestiona el comportamiento principal del "dado") y `delay()` (que implementa una función de retardo).

3.1.3. Función `main()`

```
void main(void)  
{  
    dice ();  
}
```

La función `main()` simplemente llama a la función `dice()`, que es donde ocurre toda la lógica del dado.

3.1.4. Función `dice()`

Esta es la función principal del programa y contiene la mayor parte de la lógica:

Configuración de puertos y pines:

```

GPIO = 0b00000000; // Pines en bajo
TRISIO = 0b00001000; // PIN P3 - input
ANSEL = 0b00000000; // Puertos como digitales
CMCON0 = 0b00000111; // Comparadores deshabilitados

```

- GPIO: Se configura todos los pines a bajo (0) para inicializar el estado.
- TRISIO: Se configura el pin GP3 como entrada (1).
- ANSEL: Configura todos los pines como digitales.
- CMCON0: Deshabilita los comparadores analógicos y permite que los pines de comparación sean I/O.

Variables para el manejo del "dado":

```

unsigned int timeON = 1000, timeOFF = 100;
unsigned int seed = 0;

```

- timeON y timeOFF determinan el tiempo en que los LEDs estarán encendidos o apagados.
- seed es la variable que actúa como contador pseudoaleatorio para simular los valores del dado.

Lógica de generación del dado:

```

while ( 1 ){
    seed = seed + 1;
    if (BTN != 1){
        GPIO = 0b000000; // Apaga los LEDs si el botón no está presionado
    }

    if (BTN == 1){
        if (seed == 1){
            GPIO = 0b000001; // Muestra el valor 1 en los LEDs
            delay(timeON);
            GPIO = 0b000000;
            delay(timeOFF);
            seed = 0;
            continue;
        }
        ...
        // Similar para los otros valores de "seed", mostrando un número aleatorio en LEDs
    }

    if (seed == 6){
        seed = 0; // Reinicia el contador
        continue;
    }
}

```

```
}  
}
```

- El código utiliza el valor de seed para determinar qué LEDs encender. Dependiendo del valor de seed, se muestra un patrón en los LEDs que simula el valor de un dado (del 1 al 6).
- seed se incrementa en cada ciclo del bucle. Cuando el botón (BTN) no está presionado, los LEDs se apagan. Cuando el botón se presiona (BTN == 1), se muestra un número pseudoaleatorio entre 1 y 6 utilizando los LEDs.

Restablecimiento de seed: Cada vez que se genera un número, si seed alcanza el valor 6, se reinicia a 0 para empezar de nuevo el ciclo.

3.1.5. Función de retardo (delay())

```
void delay (unsigned int tiempo){  
    unsigned int i;  
    unsigned int j;  
  
    for(i=0; i<tiempo; i++){  
        for(j=0; j<1275; j++){  
        }  
    }  
}
```

La función delay() crea un retardo utilizando dos bucles anidados. El número de iteraciones del segundo bucle se ajusta para crear el tiempo de retardo deseado. Cuanto mayor es el valor de tiempo, más largo será el retardo.

3.2. Análisis electrónico

3.2.1. Implementación del boton

Cuando la simulación del circuito se encuentra en modo estacionario, es decir, sin presionar el botón, la corriente por el PIC a través del pin 3 es casi nula, lo que se ve reflejado en que la tensión por el pin 3 es 0 V. tal y como se espera de la configuración de pull-down implementada, y eso se observa en la siguiente figura.

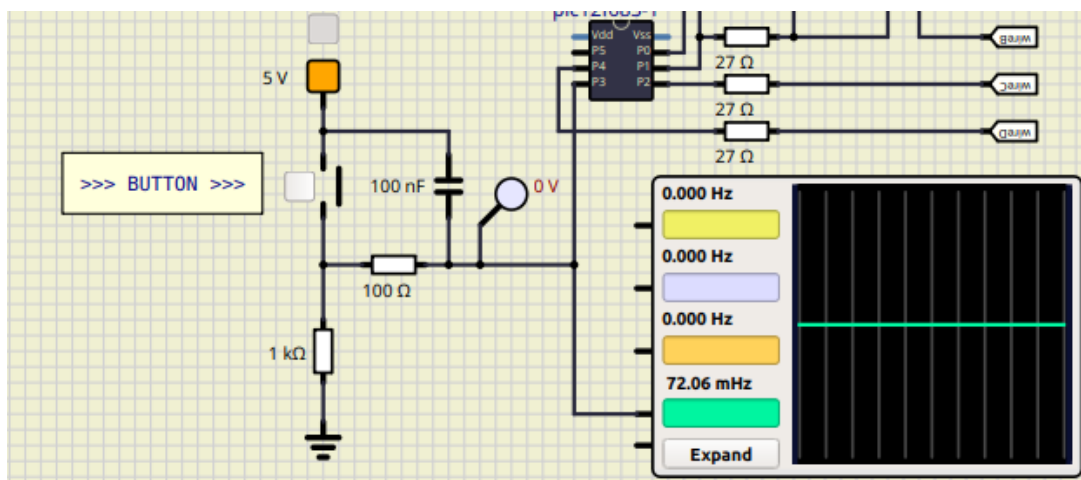


Figura 20: Botón Off

ahora bien, un dato a tener en consideración a la hora de energizar el circuito es la presencia de una respuesta transitoria, y esto es debido al efecto que provoca el capacitor, ya que se empieza a cargar.

Cuando se presiona el botón por un instante de tiempo, como se observa en la siguiente figura (foto tomada con el celular)

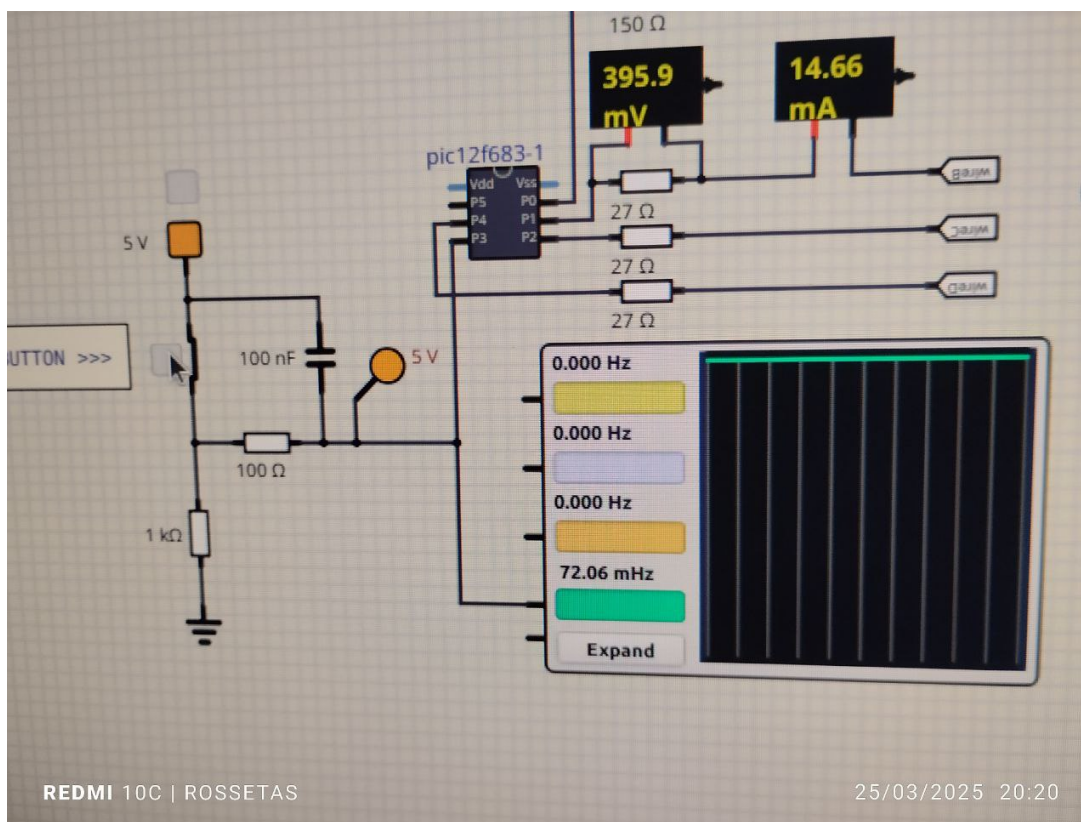


Figura 21: Botón On

se muestra la correcta implementación del botón, y se muestra el manejo de rebotes con un correcto funcionamiento; los componentes seleccionados para este caso que incluyen tanto la resistencia de $1k\Omega$ y el capacitor de $100nF$ generan una constante de tiempo τ lo suficientemente baja para funcionar correctamente.

3.2.2. Iluminacion de los leds

Para está parte de la simulación hay que tener en cuenta varios factores, lo primero y mas importante es tener bien en claro las características eléctricas de los componentes, como se menciona anteriormente; lo principal a tener en consideración es que la máxima corriente de salida que puede manejar el PIC es de aproximadamente $25mA$, y como se muestra en la siguiente figura de acuerdo a las conexiones implementadas, estas no sobrepasan esa cantidad; ahora de igual manera bajo la misma linea de trabajo, en la simulación se utilizan leds de color amarillo, los cuales tienen una corriente de operación (Forwards Current) normalmente de 20 mA y de igual forma como se observa en la figura, las corrientes no sobrepasan esa cantidad, por lo que la implementación es correcta y funcional, así mismo la resistencias seleccionadas cumplen con su labor y los resultados muestran un óptimo funcionamiento.

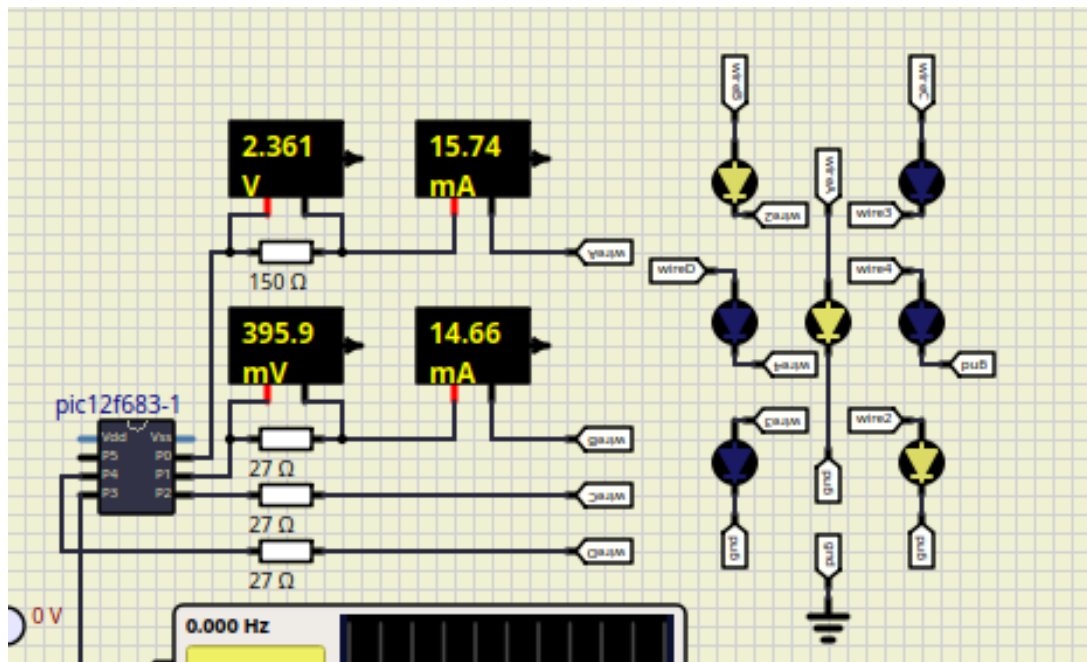


Figura 22: Mediciones eléctricas

Para efectos prácticos los Leds amarillo han sido modificados para tener una tensión de operación (Forward Voltaje) de $2V$, tratando de acerca al rango de operación real de uno de estos leds, así mismo con los valores de las resistencias en general, todos los valores de las resistencias seleccionadas son de valores comerciales reales.

3.3. Análisis de resultados

Tomando como referencia las siguientes figuras, se muestra una correcta funcionalidad del programa; de acuerdo a las conexiones implementadas, en donde por cuestiones de orden, se tiene que

hay un led conectado directamente a tierra, mientras que los demás leds tienen una conexión en serie con otro led, a eso se debe la diferencia en las corrientes.

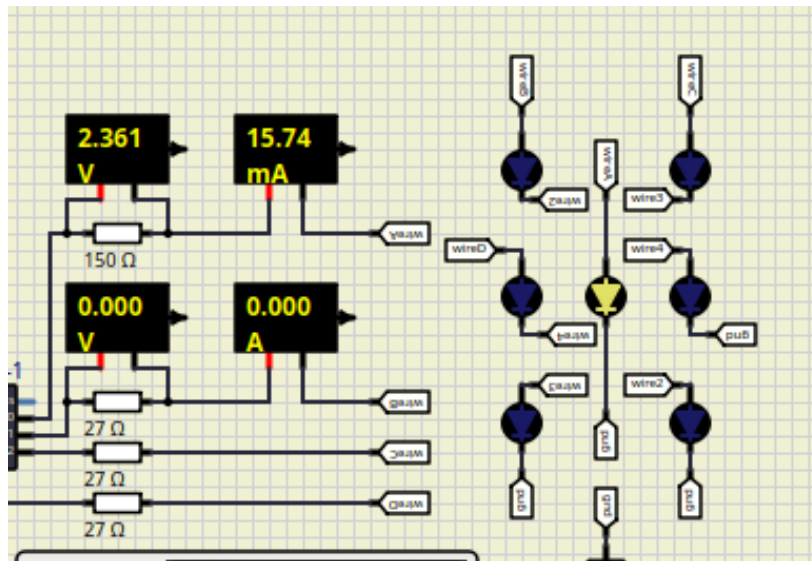


Figura 23: Medición eléctricas, pin0

La conexión del pin 0 es la que tiene una conexión solo con un led y tierra, mientras que los pines p1, p2, y p4 tiene otro led conectado y por eso, la corriente de 14.66 mA, por eso mismo solo se opta por medir las corrientes en los pines p0 y p1.

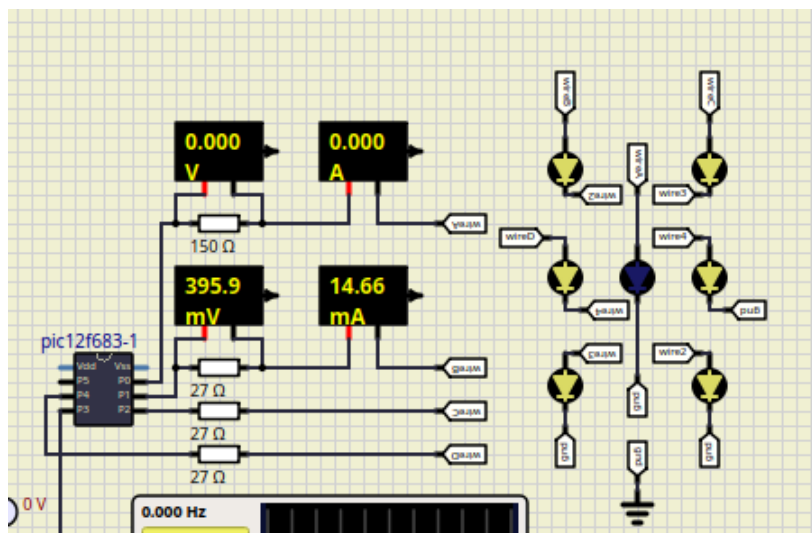


Figura 24: Medición eléctricas, (pin1, pin2, pin4)

4. Conclusiones y Recomendaciones

4.1. Conclusiones

Como parte de las conclusiones se tiene que a la hora de trabajar con un microcontrolador, es fundamental considerar tanto el software como el hardware, dado que existen aspectos claves en la hoja de fabricante que deber ser revisados cuidadosamente.

Durante la elaboración de la práctica de laboratorio, se destaca la importancia de abordar varios temas como el efecto rebote y como solucionarlo, a través de las configuraciones de pull-up o pull-down, la implementación del capacitor para evitar falsos positivos.

Asimismo, se destaca la importancia en la organización de las conexiones y la planificación previa del esquema eléctrico para evitar errores y facilitar la elaboración del trabajo.

En cuanto a la programación, se obtiene un firmware correcto que implementa un funcionamiento del simulador del Dado, un aspecto importante en como generar números aleatorios por medio de contadores internos; en como funcionan los GPIO's y los registros asociados a ello.

Finalmente se obtiene una simulación funcional que satisface cada aspecto de los requisitos, se obtiene un esquemático eléctrico funcional así como el firmware asociado para una correcta implementación.

4.2. Recomendaciones

- Siempre tener en cuenta la resistencia de protección para los LEDs para evitar daños en los componentes.
- Mantener un orden en los cables y realizar un esquema de conexiones antes de implementarlas.
- Leer cuidadosamente la hoja del fabricante para un uso correcto de los pines del microcontrolador
- Asegurar que todos los elementos estén correctamente conectados a tierra, incluso en simulaciones, para prevenir errores.
- Utilizar herramientas de medición como probes y voltímetros para verificar el correcto funcionamiento del circuito.
- Mantener un código ordenado y bien documentado para facilitar su comprensión y depuración.

Bibliografía

1. Material de clase
2. datasheet PIC12F683

5. Apéndices

5.1. Repositorio Git

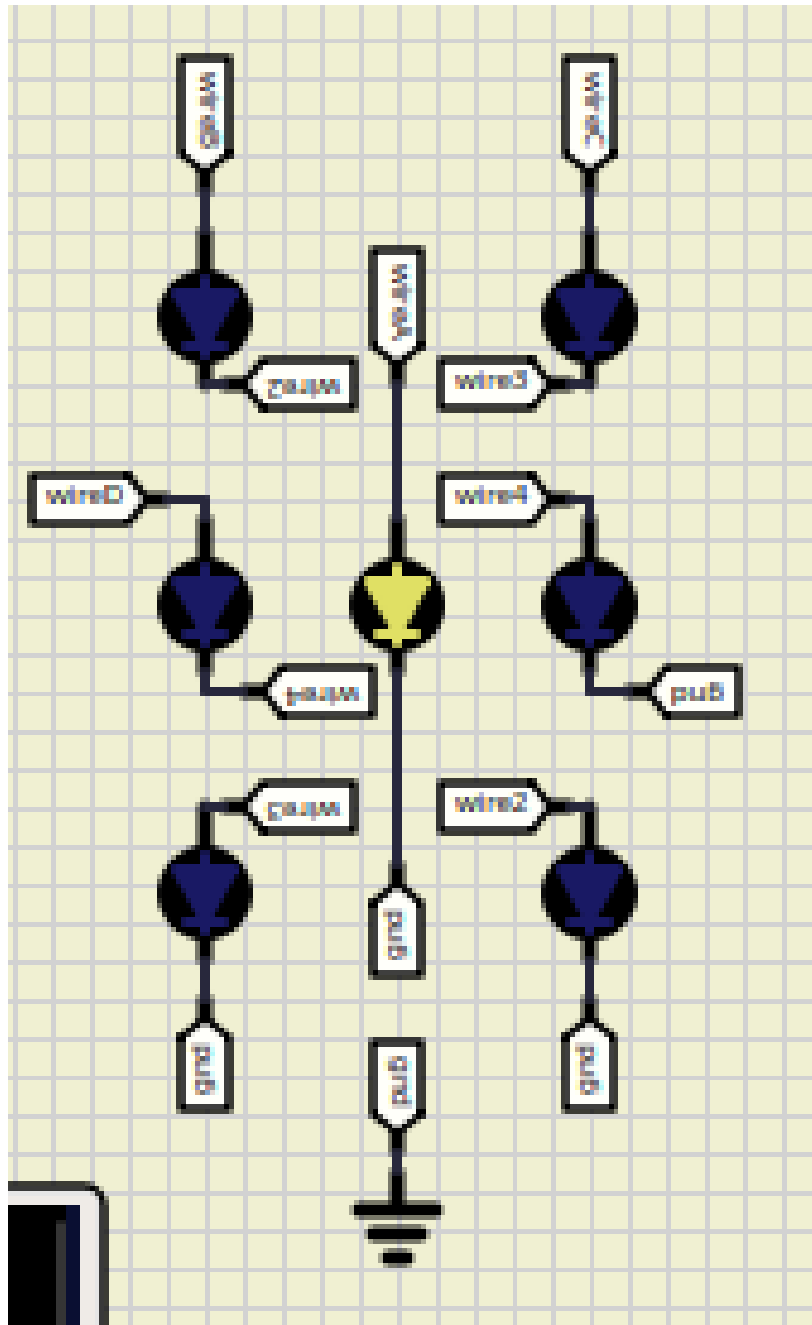
Repositorio GitHub: <https://github.com/Rossetas/MCU/tree/lab1>

5.2. Outputs

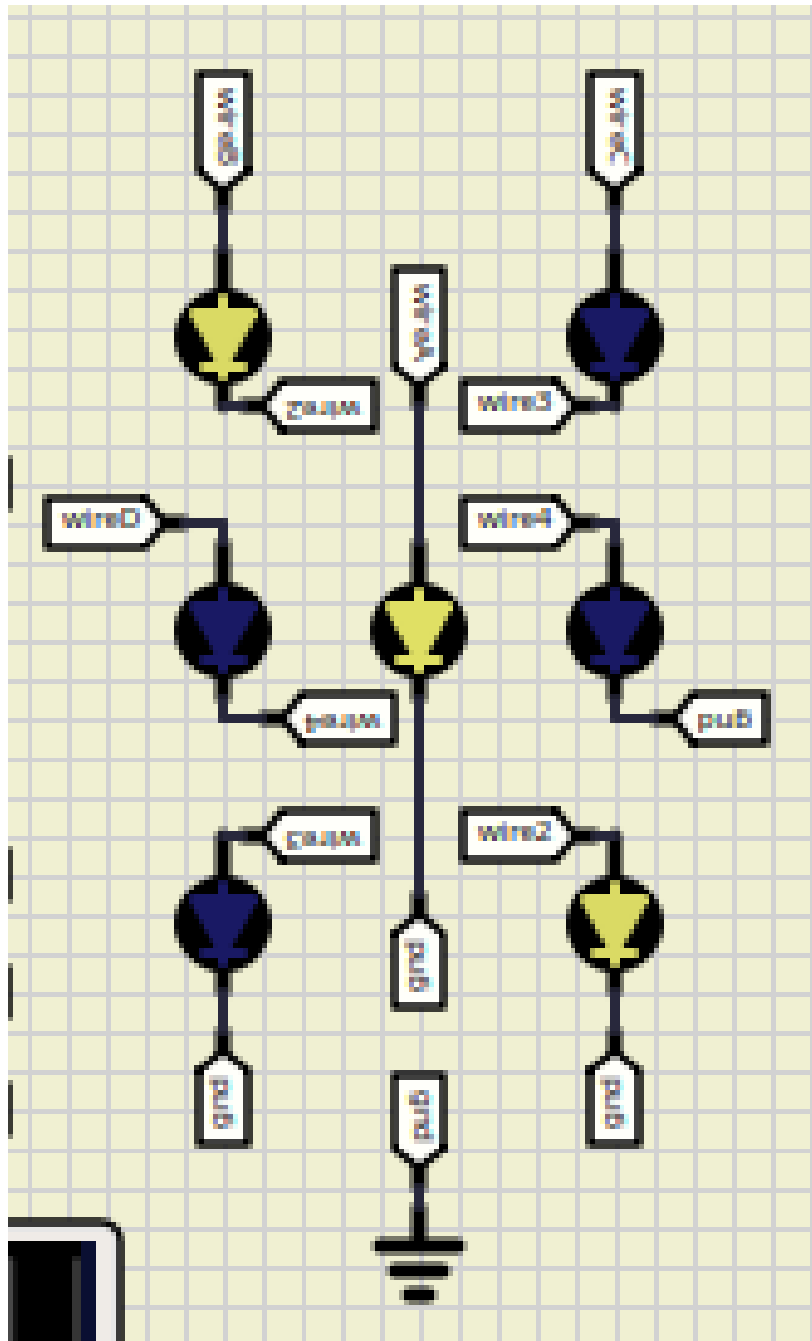


Figura 25: Caras de un dado

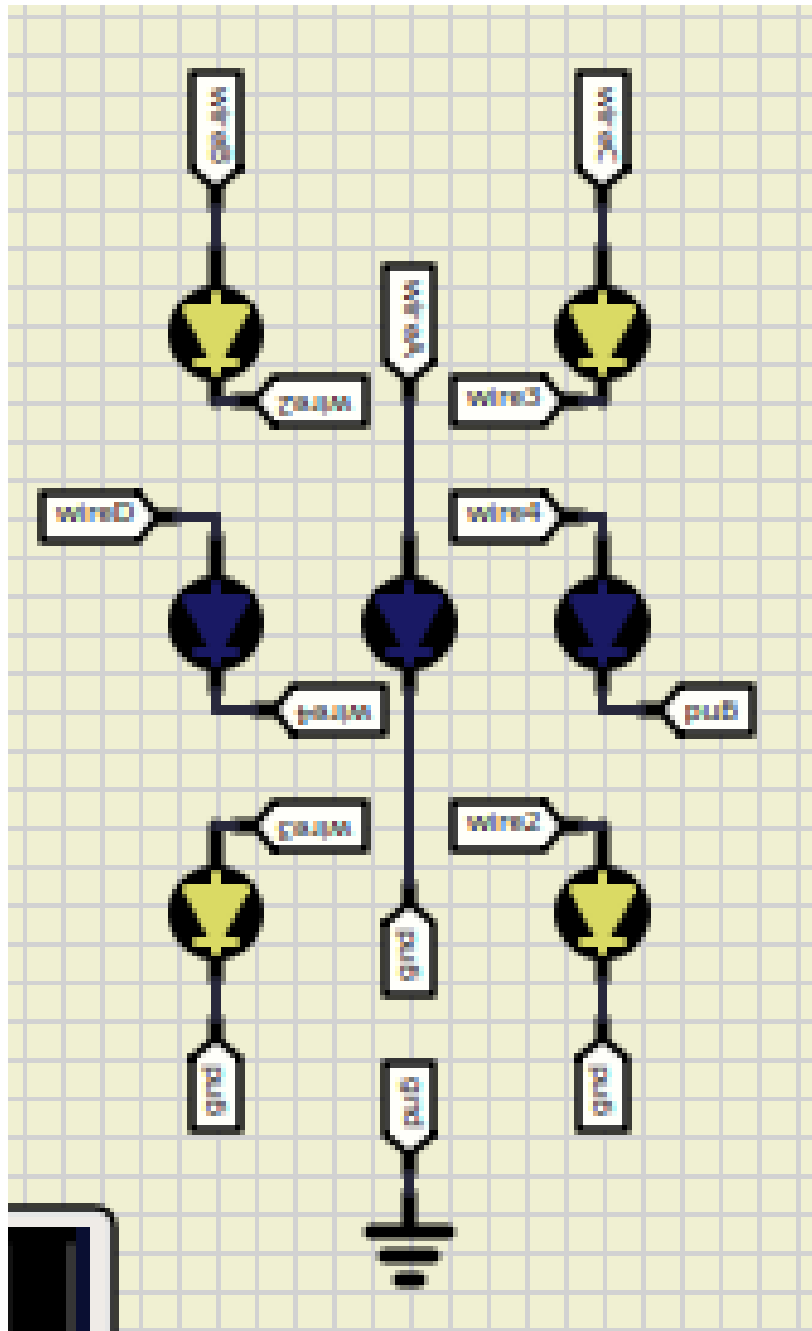
5.2.1. Cara 1 del Dado



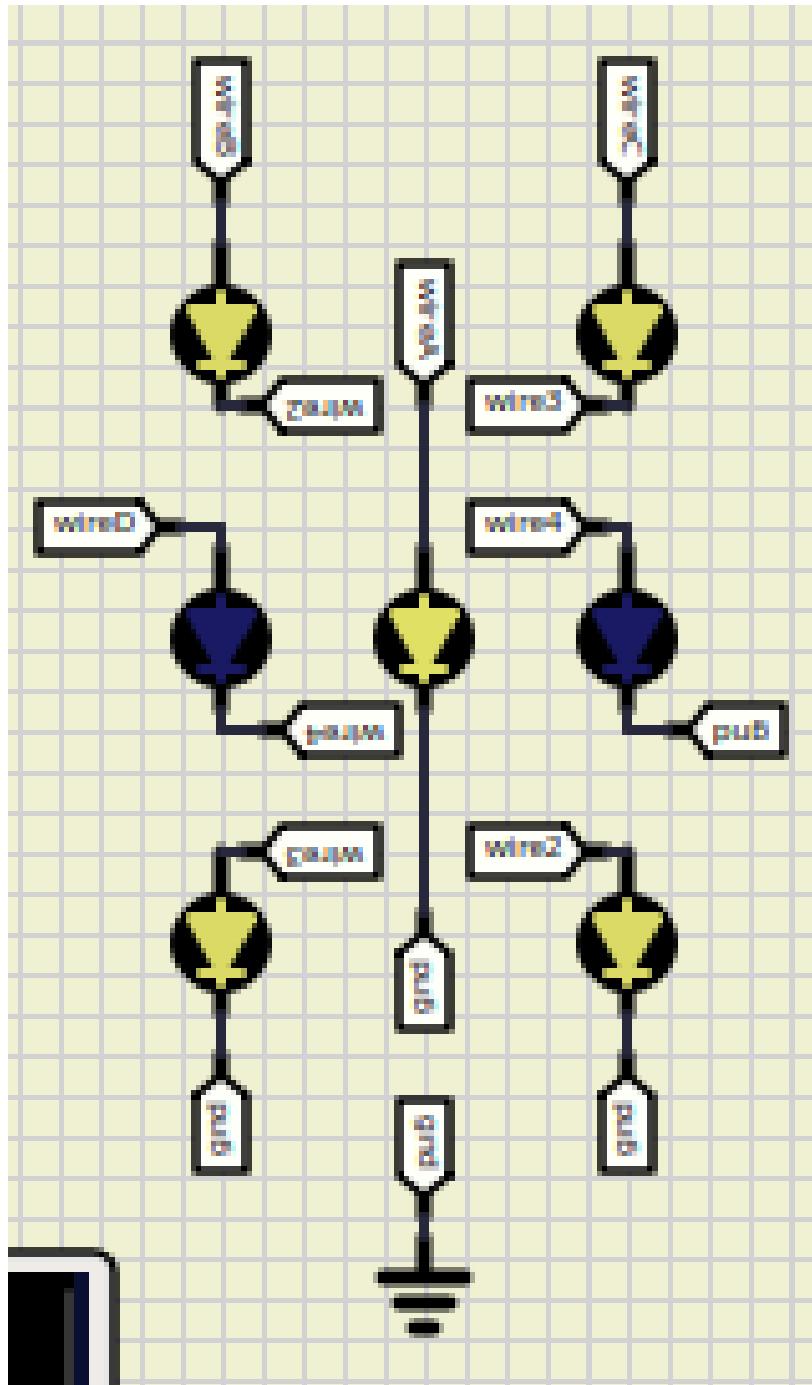
5.2.3. Cara 3 del Dado



5.2.4. Cara 4 del Dado



5.2.5. Cara 5 del Dado



5.2.6. Cara 6 del Dado

